DOKUMENTASI PBS

PoolTick

Nama	: Nazmi Mustafa
NPM	: 23312213
JobDes	: CMS, USER
Nama Kelompok	: Teluk On Top

Daftar Isi

Daftar Isi 2

CMS 3

1. Menginisialisasi aplikasi NestJS (main.ts). 3
2. Sebagai "pengikat" atau orchestrator dari semua komponen (app.module.ts). 3
3. Menangani request HTTP yang masuk ke CMS (app.controller.ts) 4
4. Konfigurasi Alamat API 4
5. Bagian Mengambil Data 4
6. Bagian Mengirim/Menyimpan Data 5
7. Bagian Proses Pembelian 6
8. Bagian Penghapusan Data 6
9. Halaman Login 7
10. Navigator Header 8
11. Stat Card (Summary Dashboard) 9
12. Export Laporan Keuangan (Excel) 10
13. Form Identitas Pelanggan 12
14. Form Kelola Katalog Tiket 13
15. Modul Katalog Tiket & Kasir 14
16. Modul Riwayat Transaksi 15
17. Struktur Proyek: Modularitas & Maintenance 16

USER 18

CMS

1. Menginisialisasi aplikasi NestJS (main.ts).

```
1  import { NestFactory } from '@nestjs/core';
2  import { AppModule } from './app.module';
3  import { join } from 'path';
4  import * as express from 'express';
5
6  async function bootstrap() {
7    const app = await NestFactory.create(AppModule);
8
9    app.use(express.static(join(__dirname, '..', 'src', 'views')));
10
11   await app.listen(3001);
12 }
13 bootstrap();
```

Bagian baris nomor 11 “await app.listen(3001)” itu saya bedakan dari localhost dari backend. Jadi untuk memanggil CMS yaitu dengan localhost:3001.

Dan untuk bagian baris nomor 9 “express.static(join(__dirname, '..', 'src', 'views')))” ini ada bagian untuk memanggil tampilan interface dari CMS.

2. Sebagai "pengikat" atau orchestrator dari semua komponen (app.module.ts).

```
1  import { Module } from '@nestjs/common';
2  import { AppController } from './app.controller';
3  import { AppService } from './app.service';
4
5  @Module({
6    imports: [],
7    controllers: [AppController],
8    providers: [AppService],
9  })
10 export class AppModule {}
11
```

File ini digunakan sebagai Root Module atau module utama dalam aplikasi CMS Pooltick. Fungsinya sebagai pusat registrasi komponen utama aplikasi agar dapat dikelolah NestJs Container.

3. Menangani request HTTP yang masuk ke CMS (app.controller.ts)

```
1  import { Controller, Get } from '@nestjs/common';
2  import axios from 'axios';
3
4  @Controller()
5  export class AppController {
6
7    @Get('tickets')
8    async getTickets() {
9      const res = await axios.get('http://localhost:3000/tickets');
10     return res.data;
11   }
12 }
```

Ini adalah file app controller untuk di bagian baris nomor 7 "@Get('tickets')" ini adalah endpoint untuk CMS menggambil data tiket.

Dan untuk bagian baris nomor 9 "*await* axios.get('http://localhost:3000/tickets')" bahwasan ini CMS bertindak sebagai client yang melakukan request ke API/Backend.

4. Konfigurasi Alamat API

```
1  const API_TICKETS = 'http://localhost:3000/tickets';
2  const API_TRANSACTIONS = 'http://localhost:3000/transactions';
```

Ini adalah code dari folder scr-views-index.html. Code ini variabel dasar yang menentukan kemana CMS harus meminta data. Semua CRUD akan lari ke URL ini di localhost:3000 yaitu bagian API/Backend.

5. Bagian Mengambil Data

```
1  const [tickets, transactions] = await Promise.all([
2      fetch(API_TICKETS).then(r => r.json()),
3      fetch(API_TRANSACTIONS).then(r => r.json())
4    ]);
```

Code ini menggunakan fetch yang berguna sebagai menarik data mentah dari API. Karena CMS tidak memiliki database sendiri dan sangat bergantung data yang dikirim dari API.Cara

kerjanya:

- **Asynchronous Request (await):** CMS menggunakan perintah await yang artinya sistem akan menunggu sampai data dari API benar-benar terkirim sebelum mencoba menampilkannya. Ini mencegah aplikasi error karena mencoba menampilkan data yang belum ada.
- **Parallel Fetching (Promise.all):** Ini teknik tingkat tinggi. CMS mengambil data Tiket dan data Transaksi secara bersamaan (paralel). Ini membuat aplikasi terasa jauh lebih cepat daripada mengambil datanya satu per satu.
- **Data Synchronization:** Setelah data mentah (JSON) diterima, CMS langsung memprosesnya ke fungsi renderTickets dan renderTransactions untuk mengubah kode-kode komputer menjadi tabel yang bisa dibaca manusia.

6. Bagian Mengirim/Menyimpan Data

```
const method = id ? 'PATCH' : 'POST';
  const url = id ? `${API_TICKETS}/${id}` : API_TICKETS;

  await fetch(url, {
    method, headers: {'Content-Type':'application/json'},
    body: JSON.stringify({ name, price })
  });
  resetForm();
  loadData();
}
```

Code di baris ini menentukan apakah CMS sendang dalam menambahkan tiket baru yaitu dengan POST atau mengedit tiket lama yaiyu PATCH. Data dikirim dalam format JSON ke API/Backend.

Cara kerja:

- **Dynamic Method Selection:** Kode kamu sangat efisien karena menggunakan satu fungsi untuk dua kegunaan.
    - Jika ada ID, CMS menggunakan metode PATCH (Update).
    - Jika ID kosong, CMS menggunakan metode POST (Tambah Baru).
- **Data Encapsulation:** Data seperti Nama Tiket dan Harga dibungkus ke dalam format JSON.stringify. Ini adalah standar industri untuk pengiriman data antar aplikasi (dari CMS ke API).

- **Header Configuration:** Penggunaan 'Content-Type': 'application/json' memberi tahu Backend: "Woi Backend, saya kirim paket berupa data JSON, tolong dibongkar sesuai format ya!".

7. Bagian Proses Pembelian

```
for (let i = 0; i < qty; i++) {
      await fetch(API_TRANSACTIONS, {
        method: 'POST', headers: {'Content-Type':'application/json'},
        body: JSON.stringify({
          ticketId: t.id, name: buyer, price: t.price, createdAt: new Date().toISOString()
        })
      });
    }
```

Code ini adalah inti dari sisi USER/PELANGGAN. Setiap tombol “Checkout” ditekan, CMS akan mengirimkan data transaksi satu per satu ke database API/Backend.

Cara kerja:

- **Looping Transaksi:** Kode menggunakan for (let i = 0; i < qty; i++). Ini berarti jika user membeli 3 tiket sekaligus, CMS akan melakukan 3 kali pengiriman data (request) ke API secara otomatis.
- **Payload (Data yang dikirim):** CMS membungkus data berupa ticketId (ID tiket), name (Nama pembeli), price (Harga saat itu), dan createdAt (Waktu transaksi).
- **Metode HTTP POST:** CMS menggunakan metode POST ke endpoint API_TRANSACTIONS. Ini memberi instruksi ke Backend: "Tolong buatkan satu catatan transaksi baru dengan data ini."

8. Bagian Penghapusan Data

```
1  await fetch(`${API_TICKETS}/${id}`, { method: 'DELETE' });
2  fetch(`${API_TRANSACTIONS}/${id}`, {method:'DELETE'}).then(loadData);
```

Untuk bagian code ini instruksi langsung dari backend untuk menghapus data tertentu berdasarkan ID uniknya.

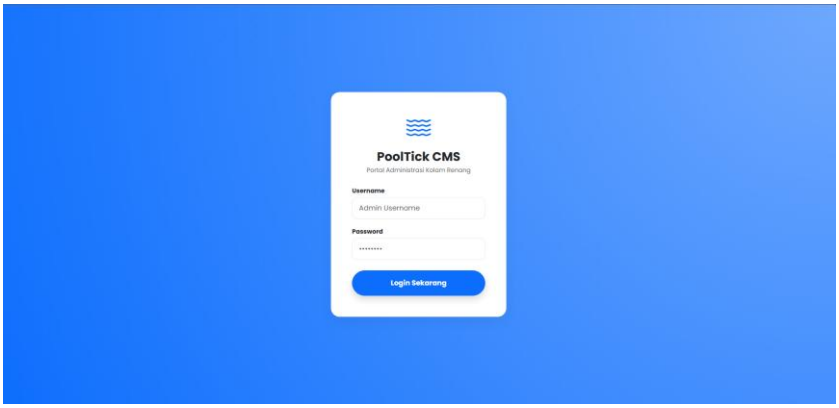
Cara kerja:

- **Metode HTTP DELETE:** CMS mengirimkan request dengan metode DELETE yang diikuti oleh ID spesifik di akhir URL-nya (contoh:

localhost:3000/tickets/5). Ini memastikan hanya data dengan ID tersebut yang terhapus.

- **Validasi Integritas:** Perhatikan kode if (trx.some(t => t.ticketId == id)). Ini adalah fitur cerdas di CMS kamu. Sebelum menghapus tiket, CMS mengecek ke API: "Apakah tiket ini pernah dipakai transaksi?". Jika iya, penghapusan ditolak agar laporan keuangan tidak error (data tiket hilang tapi riwayat transaksi masih ada).
- **Refresh Otomatis:** Setelah perintah DELETE berhasil direspon oleh API, CMS langsung menjalankan loadData(). Ini sinkronisasi agar tampilan di layar langsung berubah tanpa user harus menekan tombol refresh browser (F5).

9. Halaman Login



Halaman ini adalah pintu masuk utama untuk Admin sebelum dapat mengelola data.

- Visual Layout: Menggunakan teknik centered-card dengan latar belakang radial-gradient biru. Hal ini bertujuan untuk memfokuskan perhatian pengguna pada form input.
- Elemen UI:
    - Input Field: Dibuat dengan border-radius: 12px untuk kesan modern.
    - Action Button: Tombol "Login Sekarang" menggunakan rounded-pill dengan efek shadow agar terlihat clickable.
- UX Experience: Sistem menggunakan validasi sederhana. Jika login berhasil, halaman login akan disembunyikan (display: none) dan dashboard akan dimunculkan tanpa perlu memuat ulang browser (Single Page Experience).

```javascript
function login() {
  // Ambil nilai dari input username dan password
  const user = document.getElementById('username').value;
  const pass = document.getElementById('password').value;

  // Cek sederhana (Username: 1, Password: 1)
  if (user === "1" && pass === "1") {
    document.getElementById('login-page').style.display = "none";
    document.getElementById('dashboard').style.display = "block";
    loadData(); // Jalankan load data otomatis setelah login berhasil
  } else {
    alert("Login Gagal! Username/Pass: 1");
  }
}
```

- Data Retrieval:

  "const user = document.getElementById('username').value;"

  Program menggunakan DOM Manipulation untuk mengambil nilai tekstual yang diketikkan user secara real-time.
- Conditional Logic (Hardcoded Validation):

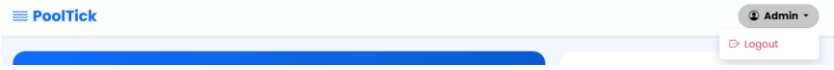
  Sistem menggunakan pengecekan berbasis kondisi

  if (user === "1" && pass === "1")

  Pada tahap pengembangan ini, kredensial masih bersifat statis (Hardcoded) untuk keperluan pengujian lokal, namun struktur fungsi sudah siap untuk diintegrasikan dengan API Authentication.
- State Transition:

  Jika validasi berhasil, sistem melakukan dua aksi sekaligus:
  1. Manipulasi DOM: Mengubah properti CSS display dari none ke block (dan sebaliknya) untuk memindahkan user dari layar login ke dashboard tanpa memuat ulang halaman (No Page Refresh).
  2. Data Initialization: Menjalankan fungsi loadData(). Ini memastikan bahwa begitu admin masuk, data tiket dan transaksi terbaru langsung ditarik dari API Backend.

10. Navigator Header



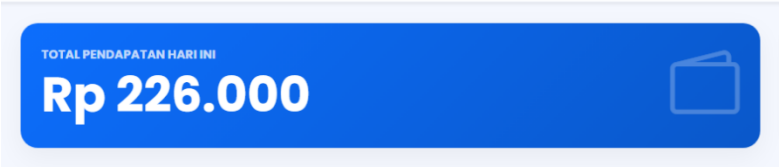
Bagian Header merupakan elemen navigasi utama yang bersifat *persistent* (tetap muncul) untuk memberikan identitas sistem dan akses cepat terhadap pengaturan akun pengguna.

1. Analisis Visual & UI (HTML & CSS)
   - Identitas Brand (navbar-brand):
     - Menggunakan kombinasi ikon ombak (bi-water) dan teks "PoolTick" dengan font tebal (fw-bold).

    - Warna: Menggunakan warna biru primer (text-primary) untuk memperkuat branding aplikasi yang berkaitan dengan fasilitas air.

- User Profile Dropdown:
    - Di sisi kanan, terdapat komponen dropdown yang dibungkus dalam tombol btn-light berbentuk kapsul (rounded-pill).
    - Menggunakan ikon profil (bi-person-circle) dan label "Admin" untuk menunjukkan identitas pengguna yang sedang aktif.
- Styling & Glassmorphism:
    - CSS Class: sticky-top shadow-sm.
    - Efek: Navbar menggunakan properti backdrop-filter: blur(10px) (efek kaca buram) dan latar belakang semi-transparan. Hal ini memastikan konten di bawah navbar tetap terlihat samar saat di-scroll, menciptakan kesan modern dan mewah.

11. Stat Card (Summary Dashboard)



Komponen ini berfungsi sebagai indikator performa utama (*Key Performance Indicator*) yang menyajikan akumulasi finansial dari seluruh transaksi yang terjadi.

1. Analisis Visual & Desain (HTML & CSS)

- Warna & Psikologi (stat-card):
    - Menggunakan gradien linear: linear-gradient(135deg, var(--primary), #0a58ca).
    - Analisis: Warna biru tua memberikan kesan kepercayaan dan stabilitas. Teknik gradien digunakan untuk memberikan efek kedalaman agar kartu terlihat lebih menonjol dibandingkan elemen lain.
- Tipografi & Kontras:
    - Teks "Total Pendapatan Hari Ini" menggunakan text-uppercase dengan opacity-75 agar fokus Admin langsung tertuju pada angka nominalnya yang lebih terang.
    - Angka nominal menggunakan class display-4 untuk memastikan keterbacaan maksimal (*high visibility*).

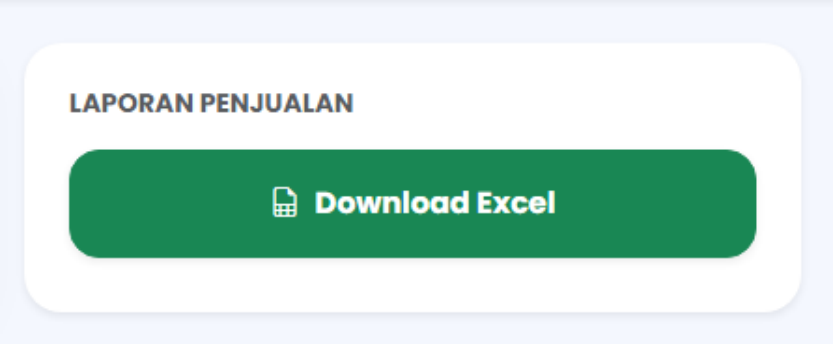
- Dekorasi Ikon:
    - Menggunakan ikon dompet (bi-wallet2) dengan ukuran sangat besar (display-1) dan opacity-25.
    - Tujuan UX: Memberikan petunjuk visual mengenai kategori kartu (Keuangan) tanpa mengalihkan perhatian dari data teks utama.

2. Analisis Logika Data (JavaScript)
   Angka yang muncul di sini bukan angka mati, melainkan hasil perhitungan dinamis.
   - Fungsi Perhitungan (renderTransactions): Di dalam file script.js (sebelumnya di bagian render), terdapat logika pemrosesan data:

```javascript
let total = 0;
transactions.map(i => {
  total += i.price;
  // ...
});
document.getElementById('total').innerText = rupiah(total);
```

   - Mekanisme Kerja:
     1. Sistem mengambil seluruh data dari API.
     2. Variabel total akan menjumlahkan setiap harga (price) dari semua objek transaksi yang ada.
     3. Hasil akhir diformat menjadi mata uang Rupiah menggunakan fungsi rupiah() sebelum ditampilkan ke elemen HTML dengan ID total.

12. Export Laporan Keuangan (Excel)



Fitur ini memungkinkan Admin untuk mengunduh seluruh data transaksi yang tersimpan di API Backend ke dalam format spreadsheet (.xlsx) yang rapi dan siap cetak.

1. Alur Kerja Pengumpulan Data (Multi-Fetch)

Sebelum file dibuat, sistem harus mengumpulkan "bahan baku" datanya dulu.

- Kode: const [dataTrx, dataTickets] = await Promise.all([...])

- Penjelasan: Sistem menarik dua data berbeda sekaligus: data Transaksi (siapa yang beli) dan data Tiket (apa yang dibeli). Teknik Promise.all digunakan agar proses penarikan data lebih cepat daripada menariknya satu-satu.

2. Transformasi Data (Data Mapping)

Data mentah dari API biasanya sulit dibaca karena hanya berisi angka/ID. Di sini sistem melakukan "penerjemahan".

- Kode: const tiket = dataTickets.find(t => t.id == trx.ticketId);
- Penjelasan: Sistem mencocokkan ticketId di setiap transaksi dengan daftar tiket yang ada.
    - Hasilnya: Di file Excel nanti yang muncul adalah tulisan "Tiket Terusan", bukan cuma angka "1". Ini meningkatkan kualitas laporan secara signifikan.

3. Penambahan Baris Total (Kalkulasi Otomatis)

Ini bagian yang kamu tanyakan tadi, kenapa ada reportData.push.

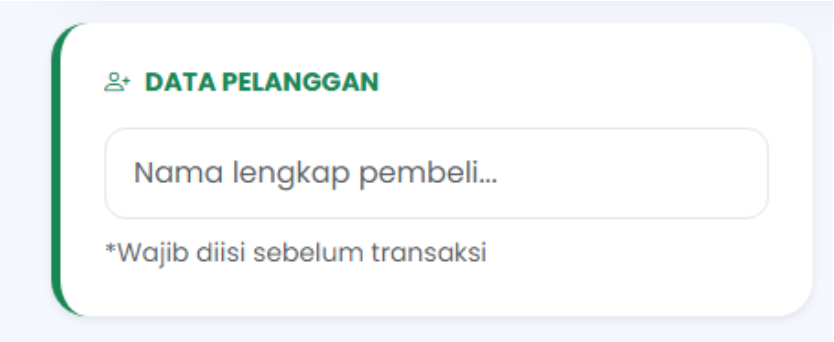
- Kode: reportData.push({ "Nama Pembeli": "TOTAL PENDAPATAN", "Harga (Rp)": totalKeseluruhan });
- Penjelasan: * Tujuan: Agar Admin tidak perlu menjumlahkan manual di Excel.
    - Cara Kerja: Sambil memproses data, sistem menghitung totalKeseluruhan. Di akhir proses, sistem menyisipkan satu baris tambahan di posisi paling bawah Excel yang berisi teks "TOTAL PENDAPATAN" dan angka total uangnya.

4. Teknis Pembuatan File (Library XLSX)

Sistem menggunakan library SheetJS untuk membungkus data menjadi file fisik.

- Worksheet (ws): Mengubah array objek JavaScript menjadi lembaran kerja Excel.
- Workbook (wb): Menciptakan file Excel baru yang bisa berisi beberapa lembar kerja.
- WriteFile: Baris terakhir XLSX.writeFile(...) adalah perintah yang memicu browser untuk otomatis mengunduh file ke komputer Admin dengan nama file unik (menggunakan new Date().getTime()).

13. Form Identitas Pelanggan



Dalam konteks operasional kolam renang, CMS ini berfungsi sebagai sistem kasir (Point of Sales) untuk melayani pembelian tiket secara langsung (on-the-spot). Berikut adalah penjelasan teknis dan fungsionalnya:

1. Peran Admin sebagai Operator

Sistem ini dirancang bukan untuk diisi oleh pelanggan, melainkan oleh Admin/Petugas Loket.

- Proses: Ketika pelanggan datang ke loket untuk membeli tiket secara tunai (cash), Admin akan menanyakan nama pelanggan dan tiket apa yang ingin dibeli.
- Input Data: Admin mengetikkan nama pada kolom "Data Pelanggan". Ini memastikan bahwa meskipun pembayaran dilakukan secara tunai, setiap lembar tiket yang terjual tetap memiliki catatan kepemilikan yang jelas di database.

2. Metode Pembayaran Cash (Tunai)

Karena ini adalah sistem manajemen internal kolam renang (CMS), proses pembayaran dilakukan secara *offline*.

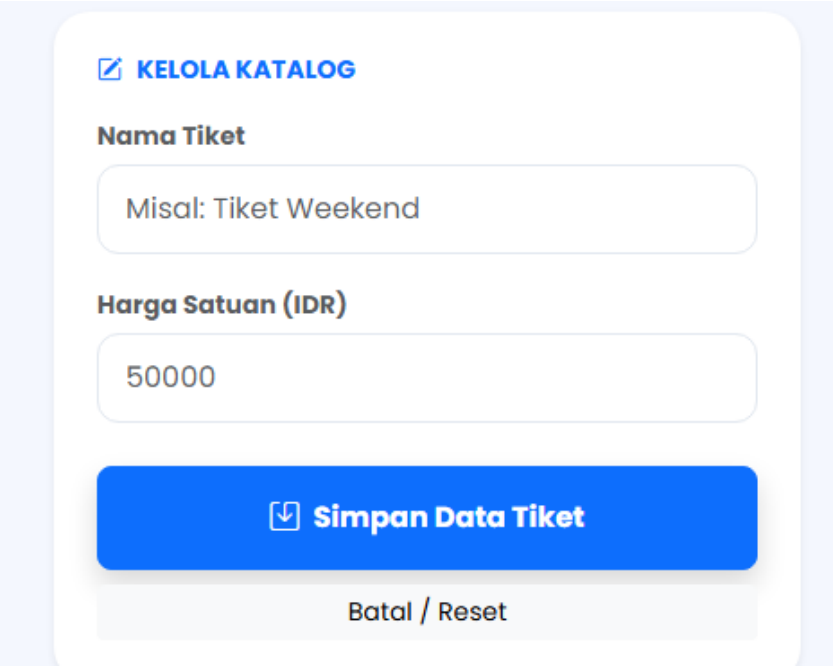
- Mekanisme: Setelah Admin menekan tombol "Checkout Terpilih", sistem menganggap pembayaran cash telah diterima oleh Admin.
- Pencatatan: Secara otomatis, sistem merekam transaksi tersebut ke dalam API (port 3000) sebagai transaksi sukses. Nominal uang yang diterima Admin akan langsung terakumulasi pada bagian "Total Pendapatan Hari Ini".

3. Keuntungan untuk Pengelola

Dengan admin yang mengisi data secara manual untuk setiap pembelian cash:

- Transparansi: Pemilik kolam renang bisa memantau apakah jumlah uang tunai di laci kasir sama dengan angka yang tertera di Total Pendapatan pada CMS.
- Keamanan: Mencegah adanya tiket yang terjual tanpa tercatat (mengurangi risiko kecurangan/kebocoran dana).
- Laporan Akurat: Saat akhir hari, Admin tinggal klik "Download Excel" untuk menyerahkan laporan penjualan harian kepada pemilik tanpa harus menghitung ulang nota manual.

14. Form Kelola Katalog Tiket



Panel ini merupakan pusat manajemen data master yang memungkinkan Admin untuk menambah atau memperbarui informasi tiket secara dinamis.

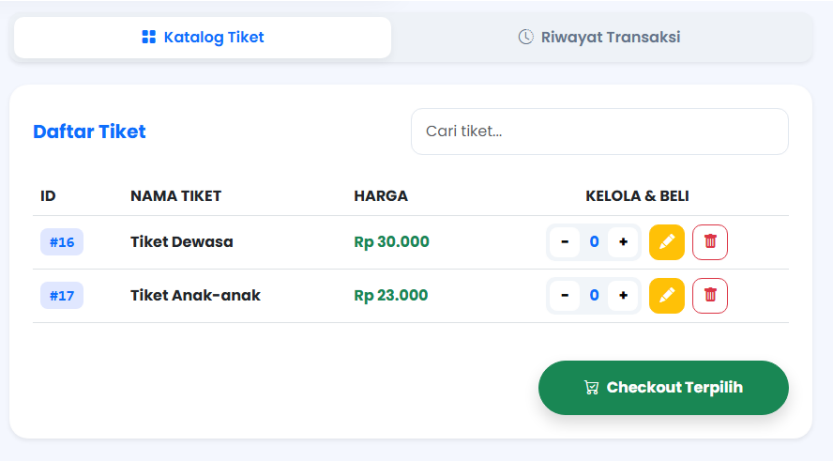
1. Komponen Antarmuka (UI)
   - Input Nama Tiket: Field teks untuk mendefinisikan kategori tiket (contoh: *Tiket Weekend, Tiket Anak-anak, VIP*).
   - Input Harga Satuan: Menggunakan atribut type="number". Secara UX, ini mencegah Admin memasukkan karakter selain angka, sehingga data yang dikirim ke database selalu berupa nilai numerik yang valid untuk perhitungan.
   - Tombol Simpan (Primary Action): Menggunakan warna biru (btn-primary) untuk menunjukkan aksi utama. Tombol ini memiliki ikon bi-save untuk memperkuat pesan visual.
   - Tombol Batal / Reset: Tombol sekunder yang berfungsi membersihkan formulir jika Admin salah input atau ingin membatalkan mode edit.
2. Analisis Logika "Dual-Mode" (JavaScript)

Salah satu kecanggihan form ini adalah kemampuannya berubah fungsi tanpa harus pindah halaman.

- Mode Create (Tambah): Secara default, form ini kosong. Ketika tombol ditekan, fungsi submitTiket() akan menjalankan perintah HTTP POST ke backend untuk membuat baris data baru.
- Mode Update (Edit): Ketika Admin mengklik ikon pensil di tabel tiket, fungsi editTiket() dipanggil.
  - Perubahan State: Data tiket lama akan "dilempar" masuk ke kotak input.
  - Visual Feedback: Tombol akan otomatis berubah warna menjadi kuning dan teksnya berubah menjadi "Perbarui Data Tiket".

    - Logika Backend: Saat disimpan, sistem akan menjalankan perintah HTTP PATCH ke ID spesifik tiket tersebut.
3. Fitur Keamanan & Validasi
    - Validation Check: Di dalam kode script.js, terdapat pengecekan if (!name || !price). Jika Admin lupa mengisi salah satu field, sistem akan memunculkan peringatan (alert) dan menolak pengiriman data. Hal ini menjamin integritas data di database backend (port 3000).
    - Reset Logic: Setelah data berhasil disimpan atau diperbarui, fungsi resetForm() akan dijalankan secara otomatis untuk mengosongkan input dan mengembalikan tombol ke kondisi awal (warna biru).

15. Modul Katalog Tiket & Kasir



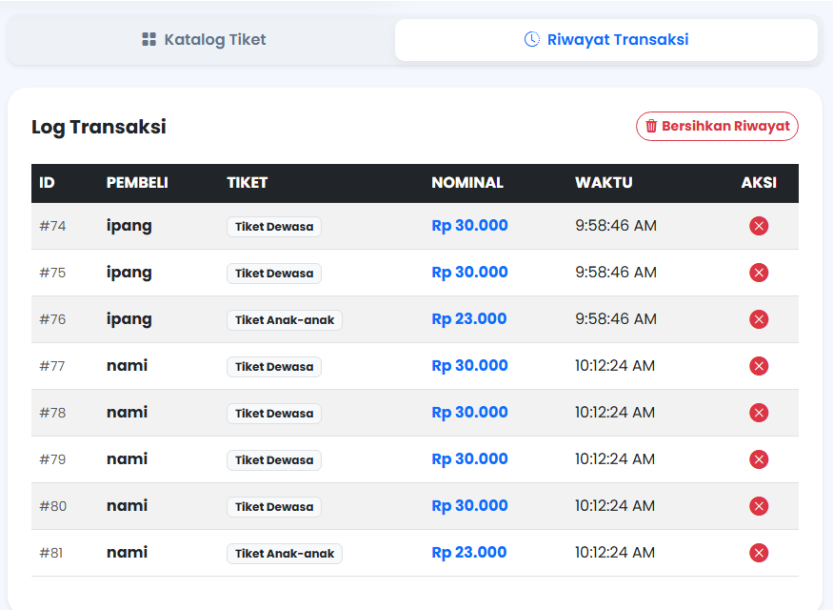
1. Fitur Pencarian Real-Time (searchTicket)
    - Implementasi: Input teks dengan atribut onkeyup="filterTable()".
    - Cara Kerja: Sistem akan memfilter baris tabel secara otomatis saat Admin mengetikkan nama tiket.
    - Tujuan UX: Mempercepat layanan jika daftar tiket sudah sangat banyak (misal: tiket rombongan, tiket event, tiket promo). Admin tidak perlu melakukan *scroll* panjang, cukup ketik keyword (contoh: "Dewasa").
2. Komponen Baris Tabel (Item Tiket)

Setiap baris tiket memiliki elemen kontrol yang sangat lengkap:

- ID Badge: Menampilkan ID unik tiket (contoh: #16) dengan font monospace agar mudah dibedakan secara visual.
- Interactive QTY Box: * Tombol plus (+) dan minus (-) memudahkan Admin menambah jumlah tiket tanpa keyboard.
    - Angka jumlah tiket (qtyMap) ditampilkan di tengah dengan warna biru agar terlihat jelas.
- Action Buttons:

    - Ikon Pensil (Kuning): Memicu fungsi editTiket() untuk mengubah harga atau nama tiket tersebut.
    - Ikon Sampah (Merah): Memicu fungsi hapusTiket(). Sistem memiliki proteksi; jika tiket tersebut sudah pernah ada transaksinya, sistem akan menolak penghapusan untuk menjaga integritas data laporan.
3. Aksi Checkout Terpilih (beliSemua)
    - Visual: Tombol besar berwarna hijau (btn-success) dengan ikon keranjang belanja.
    - Logika Bisnis:
        - Tombol ini hanya akan memproses tiket yang jumlahnya (QTY) lebih dari 0.
        - Setelah diklik, sistem akan melakukan pengiriman data ke API secara massal dan mengosongkan keranjang belanja kembali ke angka 0.

16. Modul Riwayat Transaksi



ID	PEMBELI	TIKET	NOMINAL	WAKTU	AKSI
#74	ipang	Tiket Dewasa	Rp 30.000	9:58:46 AM	
#75	ipang	Tiket Dewasa	Rp 30.000	9:58:46 AM	
#76	ipang	Tiket Anak-anak	Rp 23.000	9:58:46 AM	
#77	nami	Tiket Dewasa	Rp 30.000	10:12:24 AM	
#78	nami	Tiket Dewasa	Rp 30.000	10:12:24 AM	
#79	nami	Tiket Dewasa	Rp 30.000	10:12:24 AM	
#80	nami	Tiket Dewasa	Rp 30.000	10:12:24 AM	
#81	nami	Tiket Anak-anak	Rp 23.000	10:12:24 AM	

Modul ini berfungsi sebagai catatan digital otomatis yang merekam setiap aktivitas penjualan tiket yang telah dilakukan oleh Admin secara kronologis.

1. Navigasi & Kontrol Header
    - Tab Switcher: Bagian ini aktif saat tab Riwayat Transaksi dipilih. Antarmuka akan beralih dari daftar katalog ke daftar catatan penjualan menggunakan sistem *tabbing* Bootstrap.
    - Tombol Bersihkan Riwayat:
        - Visual: Tombol berwarna merah (outline-danger) dengan ikon sampah (bi-trash3-fill).
        - Fungsi Teknis (hapusSemuaTransaksi): Mengosongkan seluruh database transaksi di backend. Fitur ini dilengkapi dengan dialog

konfirmasi (confirm) untuk mencegah penghapusan massal data secara tidak sengaja.

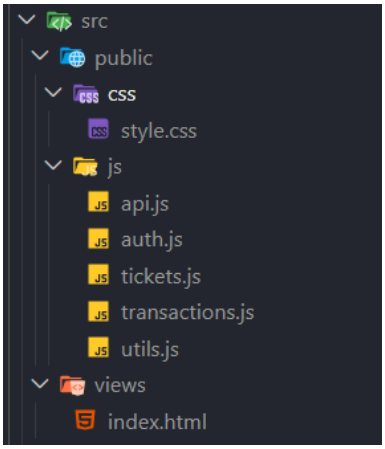
2. Struktur Tabel Log

Tabel ini menyajikan data mentah dari database dalam format yang mudah dibaca:

- ID Transaksi: Menggunakan tanda pagar (contoh: #74) sebagai identitas unik setiap tiket yang terjual.
- Nama Pembeli: Menampilkan nama pelanggan yang diinput oleh Admin pada saat checkout. Nama dicetak tebal (fw-bold) untuk memudahkan identifikasi subjek transaksi.
- Label Tiket: Menggunakan komponen *Badge* abu-abu. Secara teknis, kolom ini melakukan pencarian relasional (lookup) ke data katalog tiket untuk menampilkan nama tiket berdasarkan ID-nya.
- Nominal: Ditampilkan dengan warna biru (text-primary) untuk membedakannya dengan teks lain, menunjukkan nilai uang dari transaksi tersebut.
- Waktu (Timestamp): Menampilkan jam transaksi dengan format lokal (toLocaleTimeString). Hal ini membantu Admin melacak waktu tersibuk di kolam renang.

3. Aksi Pembatalan (Delete Single Row)
   - Ikon X Merah: Tombol aksi untuk menghapus satu baris transaksi tertentu.
   - Kegunaan: Sangat berguna jika Admin melakukan kesalahan input (misal: salah ketik nama atau salah pilih tiket) dan ingin menghapus record tersebut agar total pendapatan kembali akurat.

17. Struktur Proyek: Modularitas & Maintenance



Memisahkan logika program ke dalam file-file spesifik, sistem menjadi lebih terorganisir dan mudah untuk maintenance.

1. Folder public/css
   - style.css: Berisi seluruh aturan desain (UI), variabel warna, dan *layouting* dashboard.
   - Alasan Maintenance: Jika admin ingin mengubah warna tema aplikasi (misal dari biru ke hijau), perubahan hanya dilakukan pada file ini tanpa menyentuh logika program.
2. Folder public/js (Modular Logic)

Di sini aplikasi dipisahkan berdasarkan tugasnya:

- api.js:
  - Peran: Menyimpan konfigurasi URL (port 3000) dan fungsi dasar untuk koneksi ke server.
  - Kemudahan Maintenance: Jika alamat server berubah (misal dari localhost ke domain asli), Admin cukup mengubah satu baris di file ini.
- auth.js:
  - Peran: Khusus menangani alur login dan logout.
  - Kemudahan Maintenance: Jika ingin mengganti sistem login ke yang lebih canggih (seperti JWT atau OAuth), kodenya tidak akan bercampur dengan kode katalog.
- tickets.js:
  - Peran: Menangani manajemen katalog (Tambah, Edit, Hapus, dan Tampilan Tiket).
- transactions.js:
  - Peran: Menangani seluruh alur transaksi, perhitungan total pendapatan, riwayat, dan fitur ekspor Excel.
- utils.js:
  - Peran: Berisi fungsi pembantu (helper) seperti format mata uang Rupiah atau format tanggal.
  - Kemudahan Maintenance: Fungsi ini bisa dipanggil berulang kali di file lain (reusable), sehingga kode lebih ringkas.

3. Folder views
   - index.html:
     - Peran: Hanya sebagai kerangka (skeleton) aplikasi.
     - Kemudahan Maintenance: File ini jadi sangat bersih dan hanya berisi tag-tag HTML. Tidak ada lagi tumpukan ratusan baris skrip JavaScript di dalamnya.

USER